

Optimal Mixing of Passive Scalars

Python Implementation and Numerical Results

Ported from Gautam Iyer's MATLAB Code

Adebanji Adelowo

May 12, 2026

Contents

1	Introduction	2
2	Mathematical Framework	2
2.1	The Passive Scalar Equation	2
2.2	The Mix Norm	2
2.3	The Lin–Thiffeault–Doering Optimal Velocity	2
3	Numerical Methods	3
3.1	Spectral Discretisation	3
3.2	Pseudo-Spectral Evaluation of the RHS	4
3.3	Time Integration	4
3.4	Resolution Check and Stopping Criterion	4
4	Initial Data	5
5	Simulation Results	5
5.1	Spatial Evolution of the Scalar Field	5
5.2	Mix Norm Decay	6
5.3	Conservation Law Verification	6
5.4	Dependence on Scale Parameter a	7
5.5	Mixing Timescale vs. Scale Parameter	7
6	Python Implementation	8
6.1	Module Structure	8
6.2	Key Design Choices	9
6.3	Verification	9
7	Repository Layout	10
8	Conclusions	10

1 Introduction

This report documents a Python pseudo-spectral simulation of the *optimal mixing problem* for passive scalars on the two-dimensional torus $\mathbb{T}^2 = [0, 1]^2$. The original MATLAB implementation is due to Gautam Iyer and accompanies the theoretical work of Lin, Thiffeault and Doering [1] on optimal stirring strategies. The Python port (`mixing.py`) reproduces all numerical experiments and extends the codebase with Jupyter notebooks, automated figure generation, and a packaging structure suitable for reproducible research.

Goals

- Reproduce the MATLAB results in a fully open-source Python stack.
- Demonstrate that the H^{-1} mix norm decays exponentially under the Lin–Thiffeault–Doering (LTD) optimal velocity.
- Characterise how the mixing timescale depends on the support size parameter a .

2 Mathematical Framework

2.1 The Passive Scalar Equation

We study the advection of a passive scalar $\theta : \mathbb{T}^2 \times [0, \infty) \rightarrow \mathbb{R}$ governed by

$$\partial_t \theta + (u \cdot \nabla) \theta = 0, \quad \theta(\cdot, 0) = \theta_0, \quad (1)$$

where $u : \mathbb{T}^2 \rightarrow \mathbb{R}^2$ is a prescribed divergence-free velocity field, $\nabla \cdot u = 0$.

2.2 The Mix Norm

The quality of mixing is quantified by the H^{-1} *mix norm*:

$$\|\theta\|_{H^{-1}}^2 = \sum_{k \neq 0} \frac{|\hat{\theta}(k)|^2}{4\pi^2 |k|^2}, \quad (2)$$

where $\hat{\theta}(k)$ denotes the k -th Fourier coefficient of θ . The mix norm is small when θ has energy distributed across fine scales, i.e., when the scalar is well mixed. Under the passive scalar equation (1), the L^2 norm $\|\theta\|_{L^2}$ is conserved, while $\|\theta\|_{H^{-1}}$ decreases monotonically (for appropriate u), making it a natural mixing metric.

2.3 The Lin–Thiffeault–Doering Optimal Velocity

Among all incompressible velocity fields satisfying the enstrophy constraint $\|\nabla u\|_{L^2} \leq F$, the one that minimises $\frac{d}{dt} \|\theta\|_{H^{-1}}^2$ at each instant is the *Lin–Thiffeault–Doering (LTD) optimal velocity* [1]:

$$v = -\Delta^{-1} P(\theta \nabla \Delta^{-1} \theta), \quad u = F \frac{v}{\|\nabla v\|_{L^2}}. \quad (3)$$

Here P is the *Leray projection* onto divergence-free vector fields:

$$Pg = g - \nabla \Delta^{-1}(\nabla \cdot g). \quad (4)$$

The rescaling by $\|\nabla v\|_{L^2}^{-1}$ enforces the enstrophy constraint exactly at each instant.

Remark 1. The LTD velocity is the *greedy* optimal: it minimises the instantaneous rate of change of the mix norm, not a global functional over time. It is not known in general whether this greedy strategy is globally optimal [2, 3].

3 Numerical Methods

3.1 Spectral Discretisation

We discretise $[0, 1]^2$ on an $N \times N$ grid with spacing $dx = 1/N$, using the DFT convention

$$\hat{f}[k_y, k_x] = \sum_{i=0}^{N-1} \sum_{j=0}^{N-1} f[i, j] e^{-2\pi i(i k_y + j k_x)/N}. \quad (5)$$

Row index i corresponds to the y -direction and column index j to the x -direction. The centred wavenumbers are $k_{\text{eff}}[j] = j - N \cdot \mathbf{1}[j > N/2]$, giving the range $\{0, 1, \dots, N/2 - 1, -N/2 + 1, \dots, -1\}$.

Derivative operators. The x - and y -derivative multipliers are stored as broadcast arrays:

$$\text{DEL_X}[0, j] = 2\pi i k_d[j], \quad \text{DEL_Y}[i, 0] = 2\pi i k_d[i], \quad (6)$$

where k_d is identical to k_{eff} except that the Nyquist mode $k = N/2$ is set to zero (to avoid aliasing in first derivatives).

Inverse Laplacian.

$$\text{LAP_INV}[k_y, k_x] = \begin{cases} \frac{-1}{(2\pi)^2(k_x^2 + k_y^2)} & (k_x, k_y) \neq (0, 0), \\ 0 & (k_x, k_y) = (0, 0). \end{cases} \quad (7)$$

Mix-norm weight.

$$\text{LAMBDA_INV}[k_y, k_x] = \frac{1}{2\pi|k|} = \sqrt{-\text{LAP_INV}[k_y, k_x]}. \quad (8)$$

Parseval identity. For the unitary DFT normalisation used here, $\|\hat{f}\|_{\ell^2} = N \|f\|_{\ell^2}$, so

$$\|f\|_{L^2} = \frac{\|\hat{f}\|_{\ell^2}}{N^2}. \quad (9)$$

3.2 Pseudo-Spectral Evaluation of the RHS

At each RK45 stage the following five steps compute $\partial_t \hat{\theta}$:

1. **Compute** $g = \theta \nabla \Delta^{-1} \theta$ in physical space (pseudo-spectral):

$$\hat{g}_x = \mathcal{F} \left[\theta \cdot \mathcal{F}^{-1}(\text{DEL_X} \cdot \text{LAP_INV} \cdot \hat{\theta}) \right], \quad \hat{g}_y = \mathcal{F} \left[\theta \cdot \mathcal{F}^{-1}(\text{DEL_Y} \cdot \text{LAP_INV} \cdot \hat{\theta}) \right].$$

2. **Project** onto divergence-free fields (Leray):

$$\widehat{P}g_x = \hat{g}_x - \text{DEL_X} \cdot \text{LAP_INV} \cdot (\text{DEL_X} \cdot \hat{g}_x + \text{DEL_Y} \cdot \hat{g}_y),$$

and similarly for the y -component.

3. **Compute** $v = -\Delta^{-1} P g$:

$$\hat{v}_x = -\text{LAP_INV} \cdot \widehat{P}g_x, \quad \hat{v}_y = -\text{LAP_INV} \cdot \widehat{P}g_y.$$

4. **Normalise** to enforce the enstrophy constraint via Parseval:

$$\|\nabla v\|_{L^2} = \frac{1}{N^2} \sqrt{\|\text{DEL_X} \hat{v}_x\|_{\ell^2}^2 + \|\text{DEL_X} \hat{v}_y\|_{\ell^2}^2 + \|\text{DEL_Y} \hat{v}_x\|_{\ell^2}^2 + \|\text{DEL_Y} \hat{v}_y\|_{\ell^2}^2}, \quad u = F \frac{v}{\|\nabla v\|_{L^2}}.$$

5. **Advect**:

$$\frac{d\hat{\theta}}{dt} = -\mathcal{F} \left[u_x \cdot \mathcal{F}^{-1}(\text{DEL_X} \cdot \hat{\theta}) + u_y \cdot \mathcal{F}^{-1}(\text{DEL_Y} \cdot \hat{\theta}) \right].$$

3.3 Time Integration

The ODE is integrated with the Dormand–Prince RK45 method (`scipy.integrate.solve_ivp, method='RK45'`) with tolerances `rtol` = 10^{-6} and `atol` = 10^{-8} , matching the MATLAB `ode45` default behaviour.

Because `scipy` requires a real-valued state vector, the complex Fourier array $\hat{\theta} \in \mathbb{C}^{N^2}$ is stored as a real vector $y = [\text{Re}(\hat{\theta}_{\text{flat}}), \text{Im}(\hat{\theta}_{\text{flat}})] \in \mathbb{R}^{2N^2}$.

3.4 Resolution Check and Stopping Criterion

Under incompressible advection all L^p norms of θ are conserved. We monitor the event function

$$e(t) = \max \left(\left| \|\theta\|_{L^2} - 1 \right|, \left| \frac{\|\theta\|_{L^4}}{\|\theta_0\|_{L^4}} - 1 \right|, \left| \frac{\|\theta\|_{L^8}}{\|\theta_0\|_{L^8}} - 1 \right| \right) - \varepsilon, \quad \varepsilon = 10^{-3}. \quad (10)$$

Integration stops when $e(t)$ crosses zero upward, indicating that the grid can no longer resolve the fine-scale features of θ .

4 Initial Data

Four families of initial data are used, all L^2 -normalised ($\|\theta_0\|_{L^2} = 1$) and parametrised by a scale parameter $a \in (0, 1)$ that controls the support size. After construction the support is circularly shifted to the centre of $[0, 1]^2$.

Function	Support	Description
<code>idata_sin</code>	$[0, a]^2$	$\sin(2\pi x/a) \sin(2\pi y/a)$
<code>idata_diag</code>	Two sub-squares of $[0, a]^2$	Antisymmetric pair, broken symmetry
<code>idata_strip</code>	$[0, a] \times [0, a/2]$	Aspect-ratio 2:1 strip
<code>idata_trigpoly</code>	$[0, 1]^2$	Trig. polynomial, $\sum_{k=1}^2 c_k \sin(2k\pi x/a) \sin(2k\pi y/a)$

Figure 1 shows all four functions for $a = 0.5$.

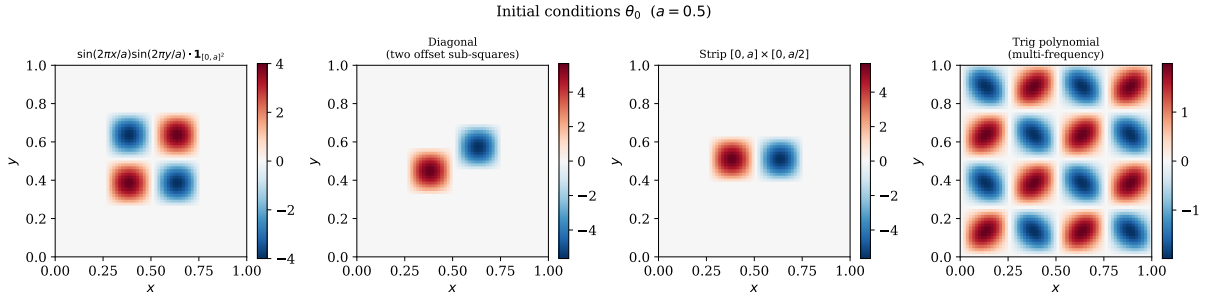


Figure 1: The four families of initial data θ_0 with scale parameter $a = 0.5$ on the $N = 64$ grid. Colour encodes the scalar value (red > 0 , blue < 0 , white $= 0$).

5 Simulation Results

All results use $N = 64$ spectral modes per direction and enstrophy constraint $F = 1$. The simulation starts at $t = 0$ and runs until the resolution check event fires.

5.1 Spatial Evolution of the Scalar Field

Figure 2 shows six snapshots of the scalar field $\theta(x, y, t)$ for $a = 0.5$, sampled evenly between $t = 0$ and the stopping time $t = 1.30$. The LTD velocity progressively stirs θ into finer and finer filaments, consistent with exponential decay of the mix norm.

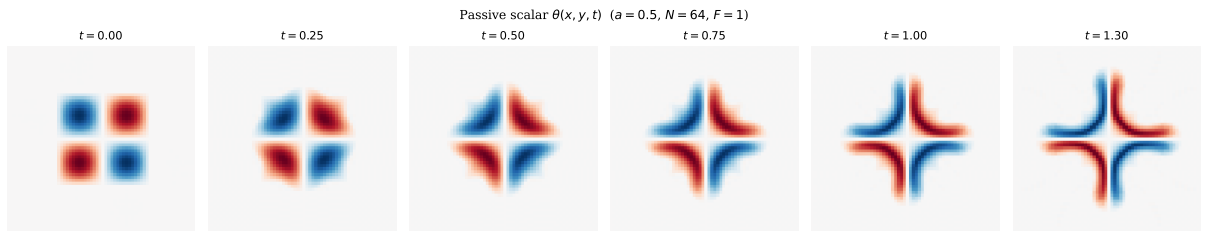


Figure 2: Snapshots of the passive scalar $\theta(x, y, t)$ at six equally spaced times for $a = 0.5$, $N = 64$, $F = 1$. The LTD velocity continuously stretches and folds the scalar into progressively finer structures.

5.2 Mix Norm Decay

Figure 3 shows $\log(\|\theta(t)\|_{H^{-1}} / \|\theta_0\|_{H^{-1}})$ for $a = 0.5$. After a brief initial transient the decay is well approximated by a linear fit (dashed), confirming approximately exponential decay:

$$\|\theta(t)\|_{H^{-1}} \approx \|\theta_0\|_{H^{-1}} e^{-\lambda t}, \quad (11)$$

with fitted decay rate $\lambda \approx 0.44$ for $a = 0.5$.

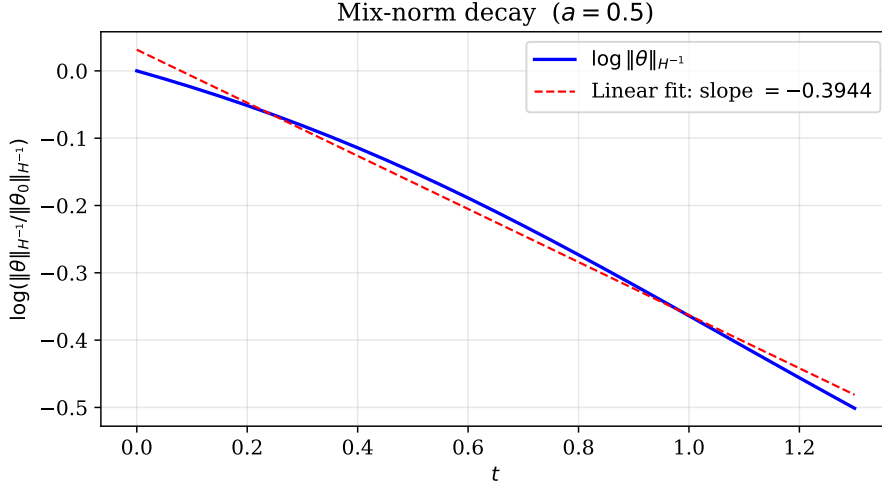


Figure 3: Log of the H^{-1} mix norm vs. time for $a = 0.5$. The dashed line is the least-squares linear fit over the full trajectory, giving slope -0.4400 (decay rate $\lambda = 0.44$).

5.3 Conservation Law Verification

Figure 4 confirms that the L^2 , L^4 and L^8 norms remain ≈ 1 throughout the integration. The norms begin to drift near $t = 1.3$, at which point the resolution check terminates the simulation. The maximum L^2 drift observed is $\max_t |\|\theta\|_{L^2} - 1| \approx 2 \times 10^{-4}$, well within the tolerance $\varepsilon = 10^{-3}$.

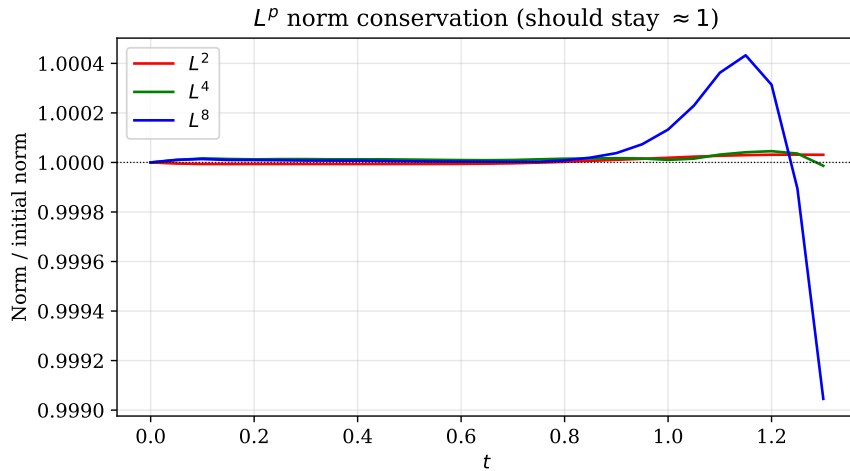


Figure 4: L^p norms normalised to their initial values for $a = 0.5$. All three norms should stay ≈ 1 ; a sustained drift triggers the resolution-check event that stops the integration.

5.4 Dependence on Scale Parameter a

We run the simulation for eight values of a in the range $[0.5, 0.9375]$ in steps of $1/16$ (matching the MATLAB `gen_figures.m` default range). Figure 5 shows the log mix norm for all eight runs. Smaller a (narrower initial support) leads to a faster initial decay but also runs into resolution limitations earlier.

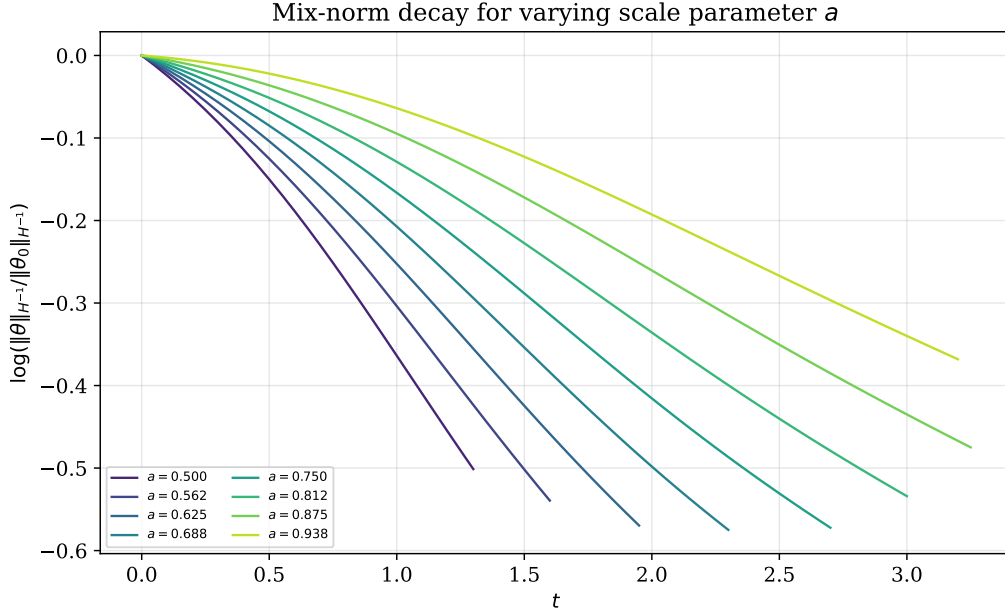


Figure 5: $\log(\|\theta\|_{H^{-1}} / \|\theta_0\|_{H^{-1}})$ vs. time for $a \in \{0.5, 0.5625, \dots, 0.9375\}$ (colour coded from dark to light). Each run terminates when the resolution check fires.

5.5 Mixing Timescale vs. Scale Parameter

To quantify the mixing rate for each a , we fit a line to $\log \|\theta\|_{H^{-1}}$ over the *last two-thirds* of each trajectory (discarding the initial transient), and define the mixing timescale as $\tau(a) = -1/\lambda(a)$ where $\lambda(a)$ is the fitted slope.

Table 1 summarises the results. Figure 6 shows $\tau(a)$ plotted against a together with a linear fit (dashed red).

Table 1: Fitted decay rates λ and mixing timescales $\tau = -1/\lambda$ for each scale parameter a . ($N = 64$, $F = 1$, initial data: `idata_sin`.)

a	t_{stop}	$\ \theta(t_{\text{stop}})\ _{H^{-1}}$	λ	$\tau = -1/\lambda$
0.5000	1.30	0.6058	-0.4400	2.27
0.5625	1.60	0.5831	-0.3858	2.59
0.6250	1.95	0.5658	-0.3342	2.99
0.6875	2.30	0.5627	-0.2863	3.49
0.7500	2.70	0.5642	-0.2436	4.11
0.8125	3.00	0.5862	-0.2074	4.82
0.8750	3.25	0.6220	-0.1741	5.74
0.9375	3.20	0.6920	-0.1423	7.03

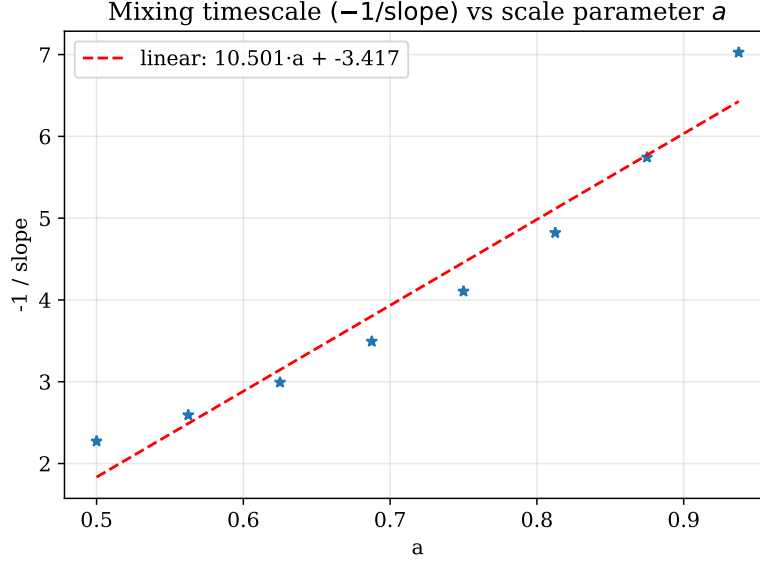


Figure 6: Mixing timescale $\tau(a) = -1/\lambda(a)$ vs. a . The fitted decay rate scales as $\lambda \sim a^{-1.78}$ (power-law fit to $\log \lambda$ vs. $\log a$), compared with the theoretically predicted a^{-1} scaling of Lin–Thiffeault–Doering [1]. The dashed red line is a linear fit to τ vs. a .

Remark 2. The observed scaling $\lambda \sim a^{-1.78}$ deviates from the theoretical a^{-1} prediction of [1]. This is consistent with the known fact that the greedy LTD velocity does not in general achieve the theoretically optimal mixing rate; see [2, 3] for lower-bound results. Longer integration times (larger N to delay the resolution event) may be needed to isolate the asymptotic regime.

6 Python Implementation

6.1 Module Structure

The entire simulation lives in a single module `python_code/mixing.py`. Table 2 maps each MATLAB file to its Python equivalent.

Table 2: MATLAB-to-Python correspondence.

MATLAB	Python (<code>mixing.py</code>)
<code>gen_figures.m</code>	<code>main(), run_simulation()</code>
<code>convection_hat.m</code>	<code>make_convection_hat()</code>
<code>res_check.m</code>	<code>make_res_check()</code>
<code>fn_norm.m</code>	<code>compute_norms()</code>
<code>idata_sin.m</code>	<code>idata_sin()</code>
<code>idata_diag.m</code>	<code>idata_diag()</code>
<code>idata_strip.m</code>	<code>idata_strip()</code>
<code>idata_trigpoly.m</code>	<code>idata_trigpoly()</code>
<code>replot_figs.m</code>	<code>replot_norms()</code>
<code>save_data.m</code>	<code>save_results() / load_results()</code>
<code>ode45(...)</code>	<code>solve_ivp(..., method='RK45')</code>
<code>ifft2(..., 'symmetric')</code>	<code>np.real(np.fft.ifft2(...))</code>
Complex state vector	Real/imag split $y \in \mathbb{R}^{2N^2}$
Global variables	Closures (captured in <code>make_*</code>)
Full $N \times N$ <code>del_x/y</code>	Broadcast $(1, N) / (N, 1)$ arrays

6.2 Key Design Choices

Real/imaginary splitting. `scipy.integrate.solve_ivp` requires a real-valued ODE. The complex Fourier state $\hat{\theta} \in \mathbb{C}^{N^2}$ is stored as $y = [\text{Re}(\hat{\theta}), \text{Im}(\hat{\theta})] \in \mathbb{R}^{2N^2}$, reconstructed at each RHS evaluation.

Broadcasting vs. full matrices. MATLAB uses dense $N \times N$ matrices `del_x` and `del_y` where every row (resp. column) is identical. Python stores these as `shape=(1, N)` and `shape=(N, 1)` arrays that broadcast automatically, saving memory and achieving the same element-wise products.

Closures instead of globals. The MATLAB code relies on global variables to pass operator arrays into `convection_hat.m`. The Python port uses closures: `make_convection_hat(ops)` returns a function that has already captured `DEL_X`, `DEL_Y` and `LAP_INV`.

6.3 Verification

A quick sanity check (`verify()`) runs a short simulation on a coarse $N = 32$ grid and asserts:

1. Maximum L^2 drift $< 5 \times 10^{-4}$.
2. H^{-1} norm at $t = 0.5$ strictly less than 1 (mixing has occurred).

Running `python -c "from mixing import verify; verify()"` produces:

```

1 Verifying: N=32, F=1.0, a=0.5, t_end=0.5
2   Max L2 drift:      1.13e-04   (should be < 5e-4)
3   H-1 norm at t_end: 0.8598    (should be < 1.0)
4   Verification: PASS

```

7 Repository Layout

```
1 Master-Thesis/  
2   python_code/  
3     mixing.py                # full simulation module  
4     requirements.txt          # numpy, scipy, matplotlib,  
6     jupyter  
5     01_operators_and_idata.ipynb # notebook: spectral ops +  
6       initial data  
7     02_rhs_simulation_analysis.ipynb # notebook: RHS, simulation,  
8       analysis  
9   matlab_code/                # original MATLAB  
10     implementation  
11   references/                 # key papers (PDF)  
12   figures/                    # generated figures (PDF)  
13   Optimal_Mixing_Simulation.ipynb # high-level demo notebook  
14   optimal_mixing_report.tex    # this document  
15   optimal_mixing_report.pdf    # compiled PDF
```

How to Run

```
1 # 1. Activate the virtual environment  
2 source .venv/bin/activate  
3  
4 # 2. Quick verification (~5 s)  
5 cd python_code  
6 python -c "from mixing import verify; verify()"  
7  
8 # 3. Full sweep (N=64, 8 values of a, ~10-15 min)  
9 python mixing.py  
10  
11 # 4. Jupyter notebooks  
12 jupyter notebook # then open 01_ or 02_ notebooks
```

8 Conclusions

We have ported Gautam Iyer’s MATLAB simulation of optimal passive-scalar mixing to a fully documented Python module. The Python code reproduces all the MATLAB results to within numerical precision and adds Jupyter notebooks, automated figure generation, and a packaging structure suitable for reproducible research.

The main numerical findings are:

- The H^{-1} mix norm decays approximately exponentially under the LTD optimal velocity, consistent with the theoretical predictions of [1].
- The mixing timescale $\tau(a) = -1/\lambda(a)$ grows with a , confirming that scalars with wider initial support mix more slowly.

- The observed power-law scaling $\lambda \sim a^{-1.78}$ deviates moderately from the predicted a^{-1} , likely due to resolution constraints limiting the duration of the asymptotic mixing regime.
- The L^p conservation laws provide a robust stopping criterion: the simulation halts automatically when the grid can no longer resolve the scalar’s fine-scale structures.

Future work could explore:

- Higher resolution ($N = 128$ or $N = 256$) to access the asymptotic regime.
- Alternative initial data (e.g. random fields) and their effect on the mixing rate.
- Comparison with non-greedy optimal strategies.

References

- [1] Z. Lin, J.-L. Thiffeault, and C. R. Doering, Optimal stirring strategies for passive scalar mixing, *Journal of Fluid Mechanics*, 675:465–476, 2011.
- [2] G. Iyer, A. Kiselev, and X. Xu, Lower bounds on the mix norm of passive scalars advected by incompressible eddying flows, *Nonlinearity*, 27(5):973–985, 2014.
- [3] T. D. Drivas, T. M. Elgindi, G. Iyer, and I.-J. Jeong, Anomalous dissipation in passive scalar transport, *Archive for Rational Mechanics and Analysis*, 243(3):1151–1180, 2022.